

코스피 주가 모니터링 시스템

개발 계획 및 구현 가이드

KIS OpenAPI + Python + 카카오 나에게 보내기

2025년 6월 | Claude Code 기반 개발 가이드

v3 | 하네스 엔지니어링 및 바이브 코딩 기법 반영

1. 시스템 개요

코스피 종목 리스트를 기반으로 실시간 주가 등락률을 조회하고, 설정한 기준치 이상 상승한 종목을 카카오톡으로 자동 알림하는 시스템이다.

1.1 핵심 기능

- KIS OpenAPI 를 통한 실시간 코스피 종목 시세 조회
- 설정 기준치(등락률) 초과 종목 자동 필터링
- 카카오 나에게 보내기를 통한 즉시 알림 발송
- Windows / Mac / Oracle Cloud 멀티 환경 지원
- settings.yaml 하나로 운영 환경 전환

1.2 기술 스택

구분	선택 기술	비고
주가 조회 API	한국투자증권 KIS OpenAPI	무료, REST API, 실시간
알림 발송	카카오 나에게 보내기	개인 사용 최적
백엔드 언어	Python 3.11+	멀티 OS 지원
운영 환경	Windows / Mac / Oracle Cloud	설정으로 전환 가능
스케줄러	Windows Task / launchd / systemd	환경별 자동 선택

2. 시스템 아키텍처

2.1 전체 흐름

```
코스피 종목 리스트 (stocks.csv)
↓
KIS OpenAPI - 실시간 시세 조회
↓
등락률 필터링 (기준치 초과 종목 선별)
↓
카카오톡 나에게 보내기 - 알림 발송
```

2.2 운영 환경 비교

항목	순수 앱 방식	서버 방식 (권장)
배터리/전원	Doze 모드 제한 있음	제한 없음
최소 조회 주기	15분 (WorkManager 한계)	1분도 가능
안정성	보통	높음
인프라 비용	없음	월 \$0~10
구현 난이도	★★★★☆	★★★★☆

※ 순수 앱 방식은 배터리 최적화 제약으로 인해 주식 모니터링처럼 짧은 주기가 필요한 경우 신뢰도가 낮다.

3. 운영 환경 선택

3.1 Oracle Cloud Free Tier

항목	내용
CPU	1/8 OCPU
RAM	1GB
스토리지	50GB
네트워크	월 10TB 아웃바운드
비용	영구 무료 (신용카드 등록 필요)

주의사항: 장기간 미로그인 시 계정/VM 회수 가능, 한국 카드 가입 거절 사례 있음

3.2 맥미니 로컬 운영

항목	내용
전기세	월 약 2,000~3,000원 (소비전력 낮음)
안정성	집 인터넷 상태에 의존

서비스 등록	launchd 설정 파일 1개 추가
코드 변경 여부	Python 코드 동일, 배포 스크립트만 다름
원격 관리	Mac 원격 로그인 설정으로 해결 가능

3.3 무료 옵션 최종 비교

서비스	무료 조건	적합도
Oracle Cloud Free Tier	영구 무료 VM	✅ 최적
AWS Free Tier	가입 후 1년만 무료	1년 후 과금 주의
Google Cloud	\$300 크레딧 (90일)	단기만 무료
집 PC (맥미니 등)	전기세만	항상 켜둬야 함

4. 프로젝트 구조

```

stock-monitor/
├── config/
│   ├── .env.example      # 환경변수 템플릿 (Git 포함)
│   ├── .env              # 실제 API 키 (Git 제외 필수)
│   └── settings.yaml     # 운영 설정 (기준치, 종목, 환경선택)
├── src/
│   ├── kis_api.py        # KIS API 인증 + 시세 조회
│   ├── stock_monitor.py  # 등락률 필터링 핵심 로직
│   ├── kakao_notify.py   # 카카오톡 메시지 발송
│   ├── scheduler.py      # 주기적 실행 관리
│   └── utils.py          # 공통 유틸 (로깅 등)
├── deploy/
│   ├── windows/setup_task.bat
│   ├── mac/setup_launchd.sh
│   └── oracle/setup_systemd.sh
├── scripts/
│   ├── setup.py          # 최초 환경설정 마법사 (대화형)
│   └── kakao_auth.py     # 카카오톡 토큰 최초 발급
├── logs/
├── stocks.csv
├── requirements.txt
├── CLAUDE.md
└── README.md

```

4.1 핵심 설정 파일 (settings.yaml)

```

environment: local_mac    # local_windows / local_mac / oracle_cloud

monitor:

```

```
interval_minutes: 3      # 조회 주기
threshold_percent: 3.0   # 알람 기준 등락률
market_open: '09:00'
market_close: '15:30'

stocks_file: stocks.csv
```

※*environment* 값 하나만 바꾸면 배포 방식(스케줄러 종류)이 자동으로 전환된다.

5. Claude Code 개발 진행 순서

5.1 CLAUDE.md 내용

Claude Code 시작 전 아래 내용으로 CLAUDE.md를 프로젝트 루트에 생성한다.

```
# Stock Monitor Project

## 프로젝트 개요
KIS OpenAPI로 코스피 종목 시세를 주기적으로 조회하고
설정한 등락률 기준 초과 종목을 카카오톡 나에게 보내기로 알람

## 기술 스택
- Python 3.11+
- KIS OpenAPI (한국투자증권)
- 카카오톡 메시지 API
- 운영환경: Windows / Mac / Oracle Cloud 모두 지원

## 운영 환경
- config/settings.yaml의 environment 값으로 환경 전환
- local_windows: Windows 작업 스케줄러
- local_mac: launchd
- oracle_cloud: systemd

## 주요 제약
- API 키는 .env에만 저장, Git 커밋 금지
- 장 운영시간 (09:00~15:30)에만 실행
- 카카오톡 토큰 자동 갱신 필수
```

5.2 단계별 프롬프트

Step 1~3: 초기 구조 생성

프로젝트 폴더 생성 후 Claude Code에 아래 프롬프트로 시작한다.

```
CLAUDE.md를 참고해서 프로젝트 초기 구조를 생성해줘.
우선 순서는:
1. .gitignore, requirements.txt, .env.example 생성
2. config/settings.yaml 생성 (환경선택 포함)
3. scripts/setup.py 대화형 초기설정 마법사 구현
```

4. src/ 모듈 순서대로 구현
(kis_api → kakao_notify → stock_monitor → scheduler)
5. deploy/ 하위 각 환경별 등록 스크립트 구현

Step 4: 카카오 메시지 발송 테스트

카카오 토큰 최초 발급은 브라우저 인증이 필요하므로 **Claude Code** 외부에서 먼저 처리한다.

순서	작업
1	https://developers.kakao.com 접속 → 앱 생성
2	REST API 키 복사
3	카카오 로그인 활성화 → Redirect URI 등록 (http://localhost:5000)
4	동의항목에서 '카카오톡 메시지 전송' 체크
5	.env에 KAKAO_REST_API_KEY 입력

scripts/kakao_auth.py를 실행해서 카카오 토큰을 발급받을게.
 발급 완료 후 kakao_notify.py로 테스트 메시지 전송해서 동작 확인해줘.
 메시지: '☑ 카카오 알림 테스트 성공 - Stock Monitor 연동 완료'

Step 5: 통합 테스트 (장중에 진행)

사전 준비 (전날 밤 또는 당일 8시 이전):

stocks.csv에 테스트용 종목 10개 추가해줘.
 테스트용으로 threshold_percent를 0.1로 낮춰서
 거의 모든 종목에 알림이 뜨도록 설정해줘.

장 시작 후 실시간 테스트:

지금 장중이야. python src/scheduler.py를 수동 실행해줘.
 첫 조회 사이클 후 로그 출력 확인하고
 카카오 메시지 정상 수신 확인해줘.
 에러 있으면 바로 수정해줘. 현재 로그: [로그 붙여넣기]

테스트 완료 후 원복:

테스트 성공. threshold_percent를 운영값 3.0으로 되돌려줘.

Step 6: 운영 환경 배포

Windows:

deploy/windows/setup_task.bat 실행해서 작업 스케줄러 등록해줘.
 평일 08:50 자동 시작, 15:40 자동 종료 설정 포함.

Mac:

deploy/mac/setup_launchd.sh 실행해서 launchd 등록해줘.
 launchctl list | grep stock 으로 등록 확인.

Oracle Cloud:

deploy/oracle/setup_systemd.sh 실행해서 systemd 서비스 등록해줘.
서버 재부팅 후에도 자동 시작되도록 enable 설정 포함.

6. Claude Code 시작 전 최종 체크리스트

6.1 KIS API 준비

체크	항목	비고
☐	한국투자증권 계좌 개설 완료	비대면 가능
☐	KIS Developers 앱 등록 완료	apiportal.koreainvestment.com
☐	App Key / App Secret 발급 완료	메모 또는 .env에 준비
☐	모의투자 계정 신청 완료	실계좌 전 테스트용

6.2 카카오 API 준비

체크	항목	비고
☐	카카오 디벨로퍼스 앱 생성 완료	developers.kakao.com
☐	REST API 키 복사 완료	
☐	카카오 로그인 활성화	앱 → 제품 설정
☐	Redirect URI 등록	http://localhost:5000
☐	동의항목 '카카오톡 메시지 전송' 체크	빠뜨리면 메시지 발송 불가 ⚠

6.3 개발 환경 (Windows)

체크	항목	확인 명령
☐	Python 3.11+ 설치	python --version
☐	Git 설치	git --version
☐	Claude Code 설치	claude --version
☐	pip 최신화	python -m pip install --upgrade pip

6.4 개발 환경 (Mac)

체크	항목	확인 명령
☐	Homebrew 설치	brew --version
☐	Python 3.11+ 설치	python3 --version

☐	Git 설치	git --version
☐	Claude Code 설치	claude --version

6.5 종목 리스트 및 운영 설정

체크	항목	비고
☐	모니터링 종목 리스트 확보	종목코드(6자리) 기준
☐	stocks.csv 형식 결정	종목코드, 종목명
☐	테스트용 소수 종목 선별	삼성전자 등 대형주 5~10개
☐	1차 운영 환경 결정	Windows / Mac / Oracle 중 선택
☐	알림 기준 등락률 결정	예: 3%, 5%
☐	조회 주기 결정	예: 3분, 5분

6.6 Git / GitHub 준비

체크	항목	비고
☐	GitHub 레포지토리 생성	Private으로 생성 권장 ⚠
☐	.gitignore에 .env 포함 확인	API 키 유출 방지
☐	git config user.email / user.name 설정	

7. Gemini 제안 방식 비교 검토 및 반영 사항

Gemini가 제안한 단일 파일 통합 방식과 본 가이드의 모듈 분리 방식을 비교 검토하고, 유효한 아이디어를 반영하였다.

7.1 전체 구조 비교

항목	Gemini 방식	본 가이드 방식
구조	단일 파일 통합 예제	모듈 분리 (5개 src 파일)
환경 지원	단일 환경	Windows / Mac / Oracle 멀티 환경
설정 관리	코드 상단 하드코딩	.env + settings.yaml 분리
API 키 보안	코드에 직접 노출 ⚠	.env 분리, Git 제외
토큰 자동 갱신	언급만 함	자동 갱신 로직 포함 설계
Mock 테스트	내장 ✅	v2에서 추가 반영
WebSocket 확장	언급 ✅	v2에서 주석으로 안내 추가

공휴일 처리	없음	포함
알림 중복 방지	1회 후 종료 구조	당일 재발송 방지 로직
로그 관리	없음	날짜별 자동 관리
장기 운영 적합성	낮음 (프로토타입 수준)	높음 (운영 목표 설계)

7.2 Gemini 방식의 장점

① Mock 데이터 설계

실제 API 없이도 바로 실행 가능한 Mock 구조를 코드에 내장하였다. `mock_base_price`를 점진적으로 하락시켜 조건 도달 시나리오를 시뮬레이션하는 방식은 개발 초기 로직 검증에 매우 실용적이다. → v2에 반영

② KIS WebSocket API 확장 방향 제시

실시간 체결 데이터가 필요한 경우 REST API 폴링 방식에서 KIS WebSocket API로 확장하는 방향을 명시하였다. 본 가이드에 누락된 내용이었다. → v2에 주식 안내로 반영

③ 카카오 메시지 템플릿 구조

`template_object` 구조와 버튼 링크 설정까지 구체적으로 제시하였다. Claude Code 프롬프트에 '버튼 링크 포함 템플릿 구조 사용' 명시로 반영한다.

7.3 Gemini 방식의 한계

한계	내용
운영 환경 고려 없음	단발성 스크립트 수준. 장기 운영에 부적합
API 키 하드코딩	보안 취약. Git 실수 시 키 유출 위험
1회 알림 후 종료 구조	실제 모니터링은 종일 동작해야 하므로 구조적으로 맞지 않음
Rate Limit 수치 오류	'초당 3~5회'로 기술했으나 KIS API 실제 제한은 초당 약 20건으로 훨씬 여유 있음

7.4 v2 반영 내용 및 추가 Claude Code 프롬프트

반영 항목	반영 위치
Mock 모드 추가	<code>settings.yaml</code> + <code>kis_api.py</code>
WebSocket 확장 주석	<code>scheduler.py</code>
카카오 템플릿 버튼 링크	<code>kakao_notify.py</code>

Mock 모드 추가 프롬프트:

```
settings.yaml에 mock_mode: true/false 옵션 추가해줘.
```



```
mock_mode: true 일 때는 실제 KIS API 호출 없이  
가상 시세 데이터로 전체 로직을 테스트할 수 있게 해줘.  
가상 데이터는 기준가 대비 랜덤 등락률을 생성해서  
실제 장치럼 동작하도록 구현해줘.
```

WebSocket 확장 주식 추가 프롬프트:

```
scheduler.py에 주식으로  
'실시간 고도화 시 KIS WebSocket API 연동으로 교체 가능'  
위치를 표시해줘.
```

카카오 메시지 템플릿 프롬프트:

```
kakao_notify.py에서 메시지 발송 시 template_object 구조를 사용하고  
버튼 링크(KIS 개발자센터 URL)가 포함되도록 구현해줘.
```

※Mock 모드를 활용하면 KIS API 키 발급 전에도 전체 로직 개발과 테스트가 가능하여 개발 속도가 크게 향상된다.

8. 하네스 엔지니어링(Harness Engineering) 적용

8.1 하네스 엔지니어링이란

하네스(harness)는 원래 말에 씌우는 마구를 뜻한다. 아무리 강력한 말이라도 마구 없이는 밧을 갈 수 없듯, AI 코딩 에이전트도 모델 자체의 성능보다 그것을 둘러싼 시스템(규칙, 도구, 스킬, 메모리, 피드백 루프)이 결과를 좌우한다. Terraform 창시자 Mitchell Hashimoto는 "에이전트가 실수할 때마다, 그 실수가 다시는 발생하지 않도록 시스템을 엔지니어링하는 것"으로 정의하였다.

실제 효과 사례: LangChain은 동일 모델로 하네스만 개선해 벤치마크 순위를 30위권 밖에서 Top 5로 끌어올렸으며, OpenAI Codex 팀은 사람이 한 줄도 직접 작성하지 않고 100만 줄 이상의 프로덕션 애플리케이션을 구축하였다(수작업 대비 약 1/10 시간 소요).

8.2 개념 발전 흐름

단계	연도	초점
프롬프트 엔지니어링	2023	무엇을 시킬 것인가 (단발성 지시 최적화)
컨텍스트 엔지니어링	2025	무엇을 보여줄 것인가 (한 세션의 배경)
하네스 엔지니어링	2026	어떤 작업 환경과 구조를 마련할 것인가 (영구적인 시스템)

8.3 핵심 구성요소 3 가지

- 제약: AI 에이전트가 할 수 있는 일을 제약 (아키텍처 경계, 종속성 규칙)
- 지시: 에이전트에게 무엇을 해야 하는지 알려줌 (컨텍스트 엔지니어링, 문서화)
- 검증: 에이전트가 올바르게 수행했는지 검증 (테스트, 린팅, CI 검증)

8.4 stock-monitor 프로젝트 적용 방안

① CLAUDE.md 를 워크플로우 프로토콜로 강화

기존 CLAUDE.md는 프로젝트 정보(기술스택, 운영환경)만 담고 있었다. 모든 작업에 일관 적용되는 절차 규칙을 추가한다.

```
## 📋 워크플로우 프로토콜
모든 기능 구현 요청 시:
1. 분석: 관련 코드/문서 탐색
2. 설계: 계획 작성 → 사용자 승인 (승인 없이 구현 시작 금지)
3. 구현: 단계별 빌드 + 테스트 확인 후 커밋
4. 검증: 완료 기준 체크 + 결과 보고

## 금지 규칙
- API 키를 코드에 하드코딩 금지, 반드시 .env 사용
- mock_mode 없이 실제 KIS API로 반복 테스트 금지 (rate limit 위험)
- environment 값 변경 시 배포 스크립트 재등록 안내 필수
```

② Bug Log 누적 — 가장 실전적인 적용

장중 테스트(5단계) 과정에서 겪는 에러를 CLAUDE.md에 그때그때 누적 기록한다. 다음 세션에서 같은 실수를 반복하지 않게 하는 핵심 장치다.

```
## Bug Log (재발 방지)
- 2026-06-30: KIS 토큰 발급 시 모의투자/실전투자 URL 혼동
  → URL을 settings.yaml에서 분리 관리
- 카카오 access_token 6시간 만료
  → 매 실행 시 자동 갱신 체크 로직 필수
```

③ 검증 루프(테스트) 명시

```
README.md에 '테스트 명령어' 섹션 추가해줘.
- python -m pytest tests/
코드 수정 후 반드시 이 명령어로 검증하고
결과 보고하도록 CLAUDE.md에도 명시해줘.
```

※mock 모드 기반 단위 테스트를 추가하면 자동화된 검증 루프가 생겨 안정성이 크게 향상된다.

④ 과유불급 주의 — 60 줄 제약

하네스를 과하게 쌓으면 규칙이 서로 충돌하고 컨텍스트가 비대해져 에이전트가 무엇을 따라야 할지 헷갈리는 역효과가 발생한다. stock-monitor는 소규모 개인 프로젝트이므로 워크플로우 프로토콜과 Bug Log 정도로 충분하며, CLAUDE.md를 과도하게 무겁게 만들지 않는다.

8.5 바이브 코딩 추가 기법 (하네스 외)

기법	설명	stock-monitor 적용
작은 단위로 쪼개기	큰 기능을 한 번에 시키지 않고 모듈 단	기존 4단계 진행 순서로 이미 반영

	위로 분리 요청	
승인 게이트	설계안 먼저 받고 승인 후 구현	KIS API 인증 구조 등 복잡한 부분에 적용 권장
Diff 리뷰 습관화	매 커밋 전 변경사항 직접 확인	git diff로 API 키 노출 여부 확인 습관화
재현 가능한 실패 기록	에러 발생 시 정확한 로그를 그대로 제공	기존 5단계 프롬프트에 이미 반영
점진적 신뢰 확장	읽기 권한 → 쓰기/실행 권한 단계적 확대	mock_mode → 모의투자 → 실전 투자 순서가 이 패턴에 해당

※ 가장 우선순위가 높은 적용은 Bug Log 누적이다. KIS API 인증과 카카오 토큰 갱신 과정에서 겪는 시행착오를 CLAUDE.md에 쌓아두면 동일한 실수의 반복을 구조적으로 방지할 수 있다.

9. 놓치기 쉬운 구현 포인트

항목	내용	Claude Code 명시 키워드
공휴일 처리	공휴일에도 API 호출 방지 로직 필요	'공휴일 휴장일 처리 포함'
KIS API 호출 제한	초당 약 20건 제한, 종목 많을 경우 간격 조절 필요	'rate limiting 처리 포함'
카카오 토큰 만료	엑세스 6시간, 리프레시 60일. 만료 전 재발급 알림 필요	'토큰 자동 갱신 및 만료 알림 포함'
알림 중복 방지	같은 종목 매 조회마다 알림 방지	'오늘 이미 알림 보낸 종목 재발송 금지'
로그 용량 관리	매일 쌓이는 로그 자동 삭제	'로그 30일 보관 후 자동 삭제'

10. 예상 코드 규모 및 소요시간

10.1 모듈별 코드량

모듈	언어	예상 라인
KIS API 연동 (인증 + 시세 조회)	Python	150~200줄
종목 리스트 관리 + 등락률 필터링	Python	100~150줄
카카오 나에게 보내기 연동	Python	50~80줄
스케줄러 (주기적 실행)	Python	50~80줄
설정 파일 (API 키, 기준치 등)	YAML/ENV	20~30줄
합계		약 400~500줄

10.2 현실적인 총 소요시간

구분	시간
Claude Code 코드 생성	1시간 이내
API 키 발급 + 설정 + 디버깅	반나일~1일
장중 실테스트	1회 장 운영시간
실질적 완성까지	1~2일

※KIS API 인증 디버깅과 장중 실테스트가 실제 체감 시간의 대부분을 차지한다.